Name:
St. ID:
Signature:

**BLG 223E Data Structures – CRN: 30310 (Summer 2025)**

**Midterm Exam**

**Duration: 120 Min. Provide clear answers for each question. If your answer is difficult to read or understand, you will lose points.**

**Q1) [15 pts]** Consider the following node definition for a single linked char list.

```
typedef struct Node{
    char label;
    struct Node* next;
} NODE;
```

Implement the add_to_list function with the signature below.

```
void add_to_list(NODE** list_head, char data);
```

This function adds a character at the end of the list. If the list is already empty (list_head pointer points to a NULL value), list head pointer pointed by the list_head parameter should be initalized.

```
void add_to_list(NODE** list_head, char data) {
  NODE* new_node = (NODE*)malloc(sizeof(NODE));
  new_node->label = data;
  new_node->next = NULL;

  if (*list_head == NULL) {
    *list_head = new_node;
  } else {
    NODE* current = *list_head;
    while (current->next != NULL) {
      current = current->next;
    }
    current->next = new_node;
  }
}
```

Name:
St. ID:
Signature:

**BLG 223E Data Structures – CRN: 30310 (Summer 2025)**

**Midterm Exam**

**Q2) [15 pts]** Implement the adj_mat_to_list function with the signature below.

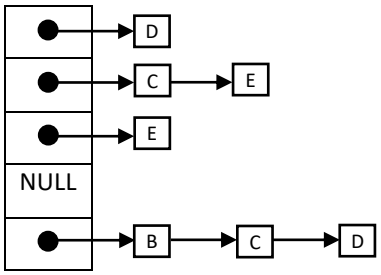
```
NODE** adj_mat_to_list(int** adj_matrix, char* node_labels, const int
num_nodes);
```

This function accepts an adjacency matrix representation of a graph over its adj_matrix parameter, transforms it to an adjacency list based representation and returns a pointer to this representation. node_labels parameters keeps each label associated with the node of the graph with the matching indice in the adjacency matrix; and num_nodes keeps the number of nodes in the graph. An example transformation is provided below.

For the parameters

```
num_nodes = 5;
node_labels = {'A', 'B', 'C', 'D', 'E'};
      adj_matrix = {{0,0,0,1,0},{0,0,1,0,1},{0,0,0,0,1},
                                            {0,0,0,0,0},{0,1,1,1,0}};
```

Function should return a structure similar to




```
NODE** adj_mat_to_list(int** adj_matrix, char* node_labels, const int num_nodes) {
  NODE** adj_list = (NODE**)malloc(num_nodes * sizeof(NODE*));

  for (int i = 0; i < num_nodes; i++) {
    adj_list[i] = NULL;
  }
  for (int i = 0; i < num_nodes; i++) {
    NODE* tail = NULL;
    for (int j = 0; j < num_nodes; j++) {
      if (adj_matrix[i][j] == 1) {
        NODE* new_node = (NODE*)malloc(sizeof(NODE));
        new_node->label = node_labels[j];
```

Name:
St. ID:
Signature:

**Midterm Exam**

```
            new_node->next = NULL;

            if (adj_list[i] == NULL) {
               adj_list[i] = new_node;
            } else {
               tail->next = new_node;
            }
            tail = new_node;
         }
      }
   }
   return adj_list;
}
```

**Q3) [10 pts]** Write the C code for a recursive function which reverses a given string. You can decide on the function parameters and how the function is called first. What is the big-O complexity of the function?

```
void reverseString(char *str, int start, int end) {
   if (start >= end)
      return;

   // Swap characters
   char temp = str[start];
   str[start] = str[end];
   str[end] = temp;

   // Recursive call
   reverseString(str, start + 1, end - 1);
}
```

Name:
St. ID:
Signature:

**BLG 223E Data Structures – CRN: 30310 (Summer 2025)**

**Midterm Exam**

**Q4) [10 pts]** Write the C code for a recursive function which determines the length of the longest sublist composed of contiguous identical elements for a given integer array. As an example, for the array int a[] = {5,7,8,8,8,8,3,3,1}, the function will return 4, since it has four "8"s as a sequence. You can decide on the function parameters and how the function is called first. What is the big-oh complexity of the function?

```
int findMaxSequence(int a[], int n, int index, int currentCount, int maxCount) {
   if (index == n)
      return (currentCount > maxCount) ? currentCount : maxCount;

   if (a[index] == a[index - 1])
      currentCount++;
   else
      currentCount = 1;

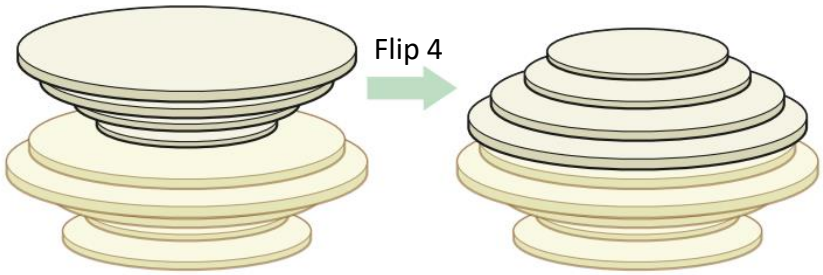
   if (currentCount > maxCount)
      maxCount = currentCount;

   return findMaxSequence(a, n, index + 1, currentCount, maxCount);
}

int longestSequence(int a[], int n) {
   if (n == 0) return 0;
   return findMaxSequence(a, n, 1, 1, 1);
}
```

**Q5 ) [10 pts]** Suppose you are given n pancakes of different sizes placed on top of each other. The only operation you can perform is a flip: insert a spatula under the top k pancakes, for some integer k between 1 and n, and flip them all over.

Write the double-linked list based data structure in C which represents the pancakes!




```
typedef struct Node {
```

Name:
St. ID:
Signature:

**Midterm Exam**

```
    int size;
    struct Node* prev;
    struct Node* next;
} Node;

Node* createNode(int size) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->size = size;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}
void append(Node** head, int size) {
    Node* newNode = createNode(size);
    if (!*head) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}
```

**Q6) [10 pts]** Write the flip function.

```
void flip(Node** head, int k) {
    if (!head || k <= 1) return;

    Node* curr = *head;
    Node* prev = NULL;
    Node* next = NULL;

    int count = 0;
    while (curr && count < k) {
        next = curr->next;
        curr->next = prev;
        curr->prev = next;
        prev = curr;
```

Name:
St. ID:
Signature:

**Midterm Exam**

```
        curr = next;
        count++;
    }

    if (*head)
        (*head)->next = curr;
    if (curr)
        curr->prev = *head;

    *head = prev;
}
```

**Q7) [10 pts]** Write the algorithm which sorts the pancakes so that smaller pancakes are on top of larger pancakes.

```
int findMaxIndex(Node* head, int n) {
    int max = -1, index = -1, i = 0;
    Node* temp = head;

    while (temp && i < n) {
        if (temp->size > max) {
            max = temp->size;
            index = i;
        }
        temp = temp->next;
        i++;
    }
    return index;
}

void pancakeSort(Node** head) {
    int n = getLength(*head);
    for (int size = n; size > 1; size--) {
        int maxIdx = findMaxIndex(*head, size);
```

```
        if (maxIdx != size - 1) {
            if (maxIdx != 0) {
                flip(head, maxIdx + 1);
            }
            flip(head, size);
        }
    }
}
```

**Q8) [10 pts]** Write an algorithm in C that counts the number of nodes in a binary tree.

```
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;


int countNodes(Node* root) {
    if (root == NULL)
        return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

**Q9) [10 pts]** Write an algorithm in C to delete all the leaves from a binary tree, leaving the root and intermediate nodes in place. You may return the updated root node.

```
int countLeaves(Node* root) {
    if (root == NULL)
        return 0;
    if (root->left == NULL && root->right == NULL)
        return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}
```